
DA6400 — Introduction to Reinforcement Learning

Name: Yash Purswani, Govind S Ashan

Due Date: March 5 2026, 11:59 PM

Student Number: ME22B214, ME23B168

Assignment: 1

Division of Work: The assignment was completed collaboratively with a clear division of tasks. **Govind Ashan** was responsible for Question 1 and Question 2.6. **Yash Purswani** was responsible for Questions 2.1, 2.2, 2.3, 2.4, and 2.5. Both members discussed the overall approach and reviewed the final results together.

The Google Drive link for the code can be found [here](#).

Problem 1 Design an MDP for a drone tasked with navigating a windy forest that has caught fire. The drone must pick up water from the lake and deliver it to the active fire zone while avoiding smoke fumes (that hinder movement and poison the water) and crashing into boulders.

1. **MDP Tuple Definition** (S, A, P, R, γ)

- **State Space** (S): Discrete grid of $5 \times 5 \times 2$ states. $s = (x, y, w)$, where $x, y \in \{0, \dots, 4\}$ and $w \in \{0, 1\}$ (0: Empty, 1: Filled)
- **Action Space** (A): $A = \{\text{North, South, East, West, Hover}\}$
- **Transition Probability Matrix** (P):
 - **Normal Cells**: $P(\text{intended}) = 0.7$, $P(\text{perpendicular}) = 0.1$ each, $P(\text{stay}) = 0.1$
 - **Smoke Cells** $\{(1, 2), (3, 2)\}$: $P(\text{intended}) = 0.4$, $P(\text{stay}) = 0.4$, $P(\text{perpendicular}) = 0.1$ each.
 - **Hovering**: $P(\text{stay}) = 1.0$ (deterministic).
- **Reward Matrix** (R): Phase-dependent (see below).
- **Discount Factor** (γ): 0.95.

Transition Subsets $P((3, 3), a, s')$ **for All Actions:**

Below we visualize the probabilities of landing in neighbors or the same cell starting from (3,3). Boundary checks are applied (off-grid moves stay in the same cell).

Table 1: Transition Probabilities $P((3, 3), a, s')$ for all actions

	Col 2	Col 3	Col 4
Row 4	0.10	0.70 (T)	0.10
Row 3	0.00	0.10 (S)	0.00
Row 2	0.00	0.00	0.00

(a) Action: North

	Col 2	Col 3	Col 4
Row 4	0.00	0.00	0.00
Row 3	0.00	0.10 (S)	0.00
Row 2	0.10	0.70 (T)	0.10

(b) Action: South

	Col 2	Col 3	Col 4
Row 4	0.00	0.00	0.10
Row 3	0.00	0.10 (S)	0.70 (T)
Row 2	0.00	0.00	0.10

(c) Action: East

	Col 2	Col 3	Col 4
Row 4	0.10	0.00	0.00
Row 3	0.70 (T)	0.10 (S)	0.00
Row 2	0.10	0.00	0.00

(d) Action: West

	Col 2	Col 3	Col 4
Row 4	0.00	0.00	0.00
Row 3	0.00	1.00 (S)	0.00
Row 2	0.00	0.00	0.00

(e) Action: Hover

Reward Subsets $R((3, 3), \text{North}, s')$ for Both Phases: The reward depends on whether the drone has water ($w = 1$) or is empty ($w = 0$).

Table 2: Phase-dependent Reward Matrices $R(s, a, s')$

	Col 2	Col 3	Col 4
Row 4	-1	-100 (B)	-1
Row 3	-1	-1	-1
Row 2	-1	-11 (S)	-1

(a) Phase 1: Empty-handed ($w = 0$)

	Col 2	Col 3	Col 4
Row 4	-1	-100 (B)	+100 (S)
Row 3	-1	-1	-1
Row 2	-1	-11 (S)	-1

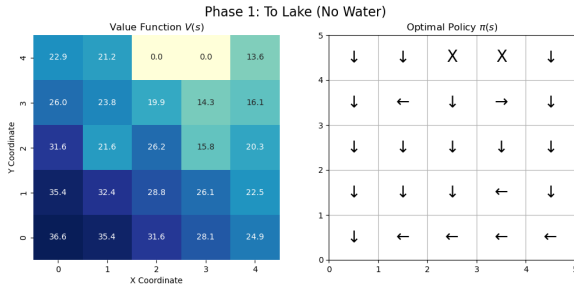
(b) Phase 2: Carrying Water ($w = 1$)

2. Value Iteration and Phase-Based Policy:

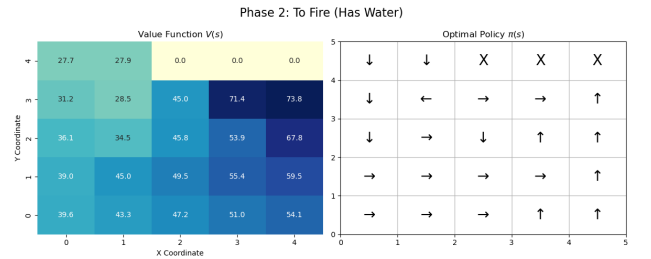
The optimal value function $V^*(s)$ and optimal policy $\pi^*(s)$ were computed using the Value Iteration algorithm with a convergence threshold of $\theta = 10^{-6}$. The solution is divided into two distinct phases based on the water status w :

- **Phase 1 (Empty, $w = 0$):** The drone's primary goal is to reach the lake at $(0, 0)$. The value function shows a gradient increasing toward the origin, and the policy directs the drone toward $(0, 0)$ while navigating around boulders at $\{(2, 4), (3, 4)\}$ and smoke at $\{(1, 2), (3, 2)\}$.
- **Phase 2 (Filled, $w = 1$):** Once water is acquired, the goal shifts to the fire zone at $(4, 4)$. The value function now peaks at the top-right corner, and the policy guides the drone toward $(4, 4)$, maintaining distance from hazards.

Visualizations :



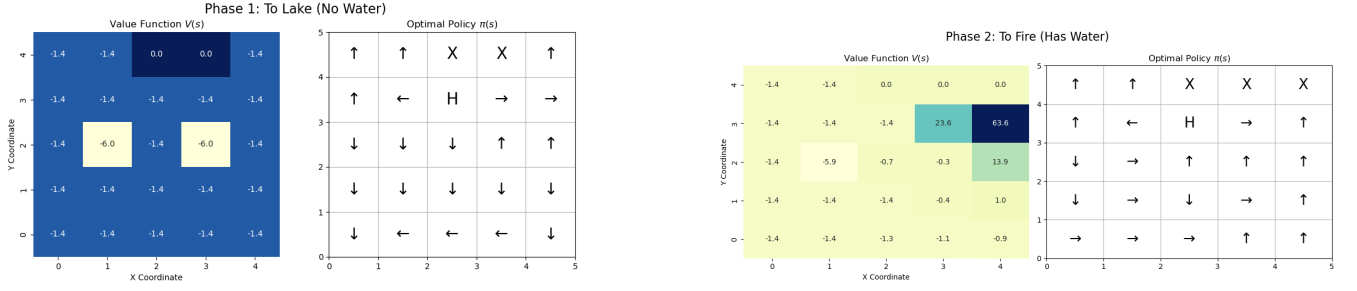
(a) Phase 1: Navigating to Lake ($w = 0$)



(b) Phase 2: Navigating to Fire ($w = 1$)

Figure 1: Optimal Value Functions and Policies for both drone phases. Heatmap intensity represents state value, and arrows indicate the optimal action $\pi^*(s)$ ($\gamma = 0.95$).

3. Suppose we change the above MDP by setting $\gamma = 0.3$.



(a) Phase 1: Navigating to Lake ($w = 0$)

(b) Phase 2: Navigating to Fire ($w = 1$)

Figure 2: Optimal Value Functions and Policies for both drone phases. Heatmap intensity represents state value, and arrows indicate the optimal action $\pi^*(s)$ ($\gamma = 0.3$).

- (a) What are the resulting optimal value function and policy? Explain the difference (if any) in the solution due to changing the problem in the two MDPs.

Ans. With $\gamma = 0.3$, the drone becomes **extremely short-sighted**.

- **Value Function:** The values decay very rapidly as we move away from the goals (Lake or Fire Zone). States more than a few steps away have nearly identical low values (around -1.4) because the discounted future reward becomes negligible.
- **Policy:** The drone focuses primarily on immediate survival. While it still navigates toward goals when very close, it becomes less determined to find the globally optimal path from far away. It often chooses to **Hover** or move cautiously if a goal is not immediately reachable.

- (b) What happens near the hazardous states when $\gamma = 0.3$? Can you explain the behaviour?

Ans. When $\gamma = 0.3$, the drone is much less likely to risk moving through hazardous states to reach a long-term goal.

- **Immediate Pain vs. Long-term Gain:** The future success payoff ($+100$) is discounted by 0.3^n (where n is the number of steps to the goal). Thus, the immediate -11 penalty for entering a smoke fume state (e.g., at $(1, 2)$ or $(3, 2)$) weighs much more heavily in comparison.
- **Result:** The drone often chooses a significantly longer path around hazardous states, or simply stops (Hover) nearby if the forward path appears too costly relative to the heavily discounted reward.

- (c) Is there any case in the two MDPs where the drone prefers doing nothing (hovering) despite the per-step penalty? Explain either ways.

Ans. Yes, for $\gamma = 0.3$, several states exhibit a preference for **Hover** despite the per-step penalty.

- **State (2,3):** In this state, the drone is surrounded by high-risk outcomes. Due to the additive wind logic, all four cardinal movement actions $\{N, S, E, W\}$ carry a non-zero probability (10% per perpendicular direction) of pushing the drone into a terminal boulder at $\{(2, 4), (3, 4)\}$ or a hazardous smoke cell at $\{(1, 2), (3, 2)\}$.
- **Risk vs. Reward:** Because $\gamma = 0.3$, the $+100$ success payoff is so heavily discounted by the distance to the fire zone that it cannot offset the immediate risk of a -100 crash or a -11 smoke penalty.
- **Optimal Choice:** For every movement action $a \in \{N, S, E, W\}$, the expected return $\sum_{s'} P(s'|s, a)[R(s, a, s') + \gamma V(s')]$ is lower than the certain -1 penalty of hovering. Consequently, the drone selects **Hover** to minimize immediate loss, effectively "giving up" on the goal to avoid a high-probability catastrophe.

- (d) Are there any interesting states (other than the ones covered above) in which the drone behaves differently in the two MDPs? Explain either ways.

Ans. A key difference appears in states that are equidistant from a hazard and a goal.

- For $\gamma = 0.95$, the drone may take a risky but shorter path through a hazardous cell because the future +100 reward is only slightly discounted.
- For $\gamma = 0.3$, the same state is treated almost like a dead end. The drone avoids central hazardous regions and prefers safer but possibly stagnant behaviour.
- **Boundary States:** Near grid edges, the $\gamma = 0.3$ drone may even move into a wall (resulting in no movement) if that is safer than progressing toward a hazard.

4. Suppose we change the MDP by penalizing the drone more for entering hazardous states (penalty = -90). Keep $\gamma = 0.95$.

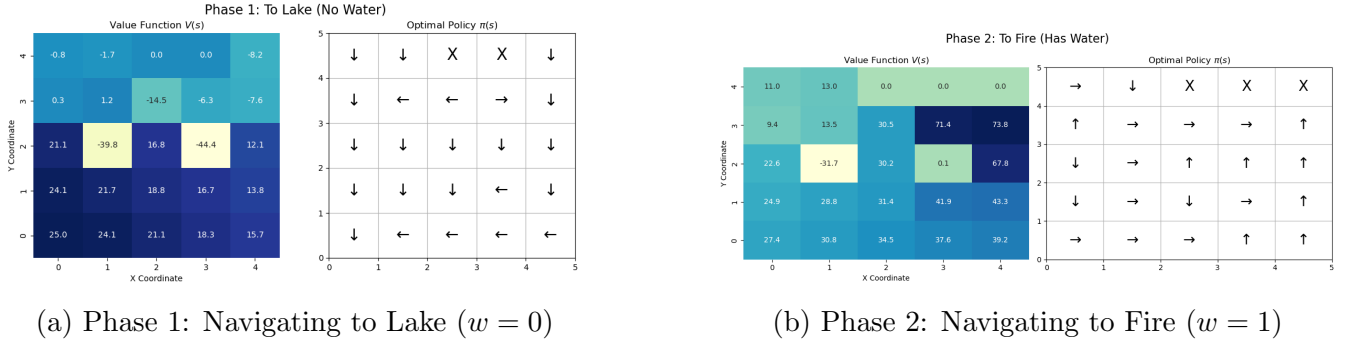
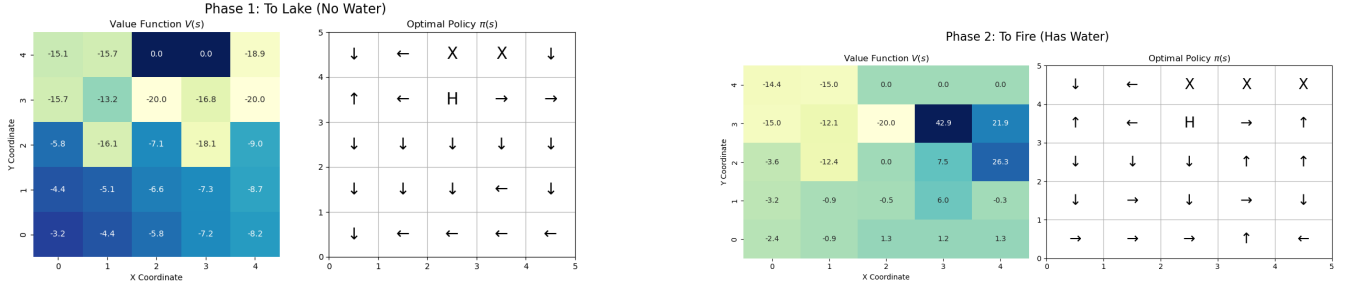


Figure 3: Optimal Value Functions and Policies for both drone phases with reward for Hazardous state as -90. Heatmap intensity represents state value, and arrows indicate the optimal action $\pi^*(s)$ ($\gamma = 0.95$).

- (a) **Optimal Value Function and Policy Comparison:** The increased penalty creates deep "value pits" at the hazardous coordinates. While the drone still seeks the +100 terminal reward, the optimal policy $\pi^*(s)$ shifts to prioritize a much larger safety buffer around smoke cells. The agent effectively treats hazards with nearly the same weight as terminal boulders.
- (b) **Preference for Hovering:** **Ans.** No, there are no states where the drone prefers to Hover in this configuration. Because $\gamma = 0.95$, the long-term value of reaching the goal remains high enough to justify the risk of movement. The cumulative cost of staying put (-1 per step indefinitely) is mathematically worse than taking a calculated, albeit stochastic, path toward the success state.
- (c) **Preferability of Longer Paths:** **Ans.** Yes, longer paths are strictly preferred when the direct route passes near boulders or hazardous cells.
- **Risk vs. Reward:** A direct path might save 2-3 steps (saving 2 to 3 units of reward), but if it carries a 20% risk of a -91 or -100 penalty, the expected value $E[V]$ drops significantly.
 - **Circuitous Navigation:** The drone will opt for "peripheral" routes—moving along the grid edges—to minimize the probability that a perpendicular wind push lands it in a high-penalty state.
- (d) **Impact of Stronger Wind (40% Success, 50% Diagonal Risk):** With the transition probabilities adjusted to 40% for the intended direction, 25% for each perpendicular direction (total 50% diagonal risk), and 10% for staying in the same cell, the agent exhibits a significant shift toward extreme risk aversion.



(a) Phase 1: Navigating to Lake ($w = 0$)

(b) Phase 2: Navigating to Fire ($w = 1$)

Figure 4: Optimal Value Functions and Policies for both drone phases with reward for Hazardous state as -90. Heatmap intensity represents state value, and arrows indicate the optimal action $\pi^*(s)$ ($\gamma = 0.95$).

Optimal Hovering at State (2,3): In the presence of 50% total perpendicular wind risk, state (2,3) shifts to a Hover ('H') policy because every cardinal movement $\{N, S, E, W\}$ carries a 25% probability of a diagonal transition into terminal boulders at $\{(2,4), (3,4)\}$ or severe hazards at $\{(1,2), (3,2)\}$. By hovering, the drone maximizes expected reward by accepting a certain -1 step penalty rather than risking a 50% aggregate chance of catastrophic penalties (-90 or -100).

Divergence Toward Boundaries and Longest Paths: The policy grids exhibit "boundary hugging" along Column 0, Column 4, and Row 0, where the drone chooses the longest path to utilize the "off-grid stay" rule as a safety mechanism. This ensures that perpendicular wind pushes result in safe collisions with the boundary rather than entering high-penalty states, mathematically justifying the increased step penalties as an acceptable trade-off for significantly higher safety.

Problem 2 The observation space is continuous. Use binning (number of bins = 10) to discretize the space (what is the number of states?). Use $\gamma = 0.99$.

1. Write Python code for SARSA and Q-learning using ϵ -greedy exploration in the above environment.

Ans. The Python implementations of both algorithms using ϵ -greedy exploration have been developed and are available in the GitHub repository. The SARSA implementation is provided in `sarsa_best.py`, and the Q-learning implementation is provided in `q_learning_best.py`. These scripts include the training loop, ϵ -decay mechanism, and evaluation procedures used in our experiments.

2. Plot return vs. time-steps/episodes curves comparing SARSA and Q-learning.

- (a) For both algorithms, tune the hyperparameters (stepsize and ϵ) and report the top three ones (resulting in highest returns). Is a constant ϵ sufficient for exploration? If not, implement an appropriate ϵ -decay schedule to get a good policy.

Ans. We performed a grid search over the learning rate (α) and exploration rate (ϵ). A high learning rate causes unstable updates and possible divergence, while a very low learning rate slows convergence. Similarly, a high ϵ leads to excessive exploration and poor convergence, whereas a very low ϵ results in insufficient exploration and suboptimal policies.

A constant ϵ is generally not sufficient, as it either over-explores or prematurely exploits. Therefore, we implemented an ϵ -decay schedule to gradually reduce exploration over time, allowing sufficient early exploration and stable convergence to a better policy. The experiments are summarized in the tables 3 and 4 below.

Table 3: Q-Learning vs SARSA Performance (Learning Rate vs ϵ)

Q-Learning			SARSA		
LR	ϵ	Score	LR	ϵ	Score
0.01	0.5	-362.23	0.01	0.3	-334.44
0.05	0.5	-421.15	0.05	0.3	-319.99
0.1	0.5	-374.76	0.1	0.3	-331.10
0.2	0.5	-428.97	0.01	0.5	-344.40
0.01	0.8	-460.95	0.05	0.5	-364.25
0.05	0.8	-475.66	0.1	0.5	-361.59
0.1	0.8	-480.16	0.01	0.8	-465.42
0.2	0.8	-489.46	0.05	0.8	-475.70
0.01	1.0	-500.00	0.1	0.8	-482.00
0.05	1.0	-499.21	0.01	1.0	-499.62
0.1	1.0	-500.00	0.05	1.0	-500.00
0.2	1.0	-499.17	0.1	1.0	-497.91

Table 4: Best Performance Comparison

Algorithm	Learning Rate	Epsilon	Best Score
Q-Learning	0.01	0.5	-362.23
Q-Learning	0.1	0.5	-374.76
Q-Learning	0.05	0.5	-421.15
SARSA	0.05	0.3	-319.99
SARSA	0.1	0.3	-331.10
SARSA	0.01	0.3	-334.44

- (b) Plot (with tuned hyperparameters) the mean performance along with confidence intervals over 10 random seeds/runs. A random seed corresponds to the initial random Q-values and the initial random start state. Explain the results along with comparing the two algorithms.

Ans. Using grid search, we found the optimal configuration for both algorithms to be learning rate ($\alpha = 0.1$) and exponential ϵ -decay with decay rate 0.9, starting from $\epsilon_{\text{start}} = 1.0$ and decaying to $\epsilon_{\text{end}} = 0.1$, refer table 5.

With this configuration, we ran 10 different random seeds and plotted the mean return along with confidence intervals. Both Q-learning and SARSA showed similar overall performance and comparable variance across runs, as seen in figure 5a indicating stable learning under the tuned hyperparameters. The confidence intervals overlapped significantly, suggesting no major performance gap between the two algorithms under this setting.

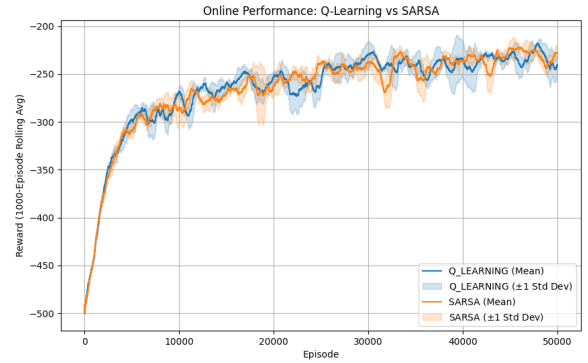
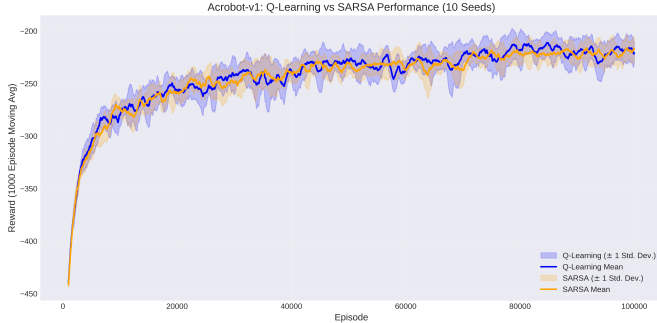
3. Run the two algorithms starting with $\epsilon = 1$, decaying ϵ till 0.1, and fixed thereafter. Compare (i) the online performance (performance while learning) and (ii) the performance of the policies after finishing learning (i.e., without any exploration). Explain the differences, if there exist any.

Ans. The experiment was conducted with learning rate $\alpha = 0.1$ and exponential ϵ -decay ($\epsilon_{\text{start}} = 1$, $\epsilon_{\text{end}} = 0.1$, decay = 0.9) over 3 random seeds. During online learning, performance was initially low due to high exploration ($\epsilon = 1$), but improved steadily as ϵ decayed. After training, policies were evaluated without exploration, resulting in higher and more stable returns compared to the online phase. As shown in Fig. 5b and summarized in Table 6, both Q-learning and SARSA exhibited very similar online and offline performance, with no significant difference observed

Table 5: Q-Learning vs SARSA Performance (Learning Rate vs ϵ Decay Rate)

Q-Learning		
LR	Decay	Score
0.1	0.9	-207.48
0.1	0.995	-250.78
0.1	0.999	-252.13
0.05	0.99	-257.40
0.05	0.999	-262.10
0.05	0.995	-264.18
0.1	0.95	-270.94
0.05	0.9	-272.09
0.2	0.999	-281.99
0.05	0.95	-292.96
0.2	0.95	-295.54
0.2	0.99	-303.52
0.01	0.9	-306.77
0.2	0.995	-311.34
0.2	0.9	-313.44
0.01	0.999	-324.39
0.1	0.99	-345.23
0.01	0.95	-367.00
0.01	0.995	-401.96
0.01	0.99	-422.82

SARSA		
LR	Decay	Score
0.1	0.9	-244.69
0.1	0.95	-262.58
0.1	0.999	-265.35
0.05	0.95	-267.38
0.05	0.999	-275.45
0.05	0.99	-276.27
0.05	0.995	-280.88
0.1	0.99	-288.76
0.05	0.9	-304.45
0.1	0.995	-319.82
0.01	0.995	-325.21
0.01	0.99	-358.01
0.01	0.999	-359.91
0.01	0.9	-363.25
0.01	0.95	-382.19
0.005	0.995	-388.35
0.005	0.999	-404.03
0.005	0.99	-409.98
0.005	0.9	-420.84
0.005	0.95	-437.20



(a) Q-Learning vs SARSA Comparison with 10 seeds

(b) On-line performance comparison of Q-Learning and SARSA

Figure 5

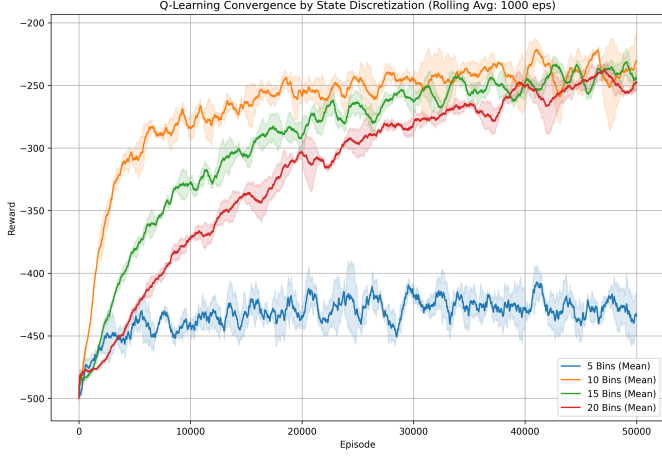
between the two algorithms under this setting.

- Intuitively, increasing the number of bins results in better state representation, enhancing granularity. This can result in learning a better policy. Is this always the case? Are there any downsides? Explain (try 5, 10, 15, and 20 bins).

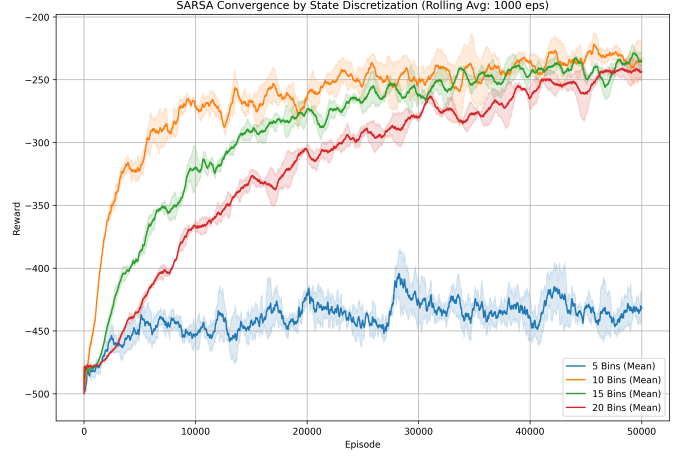
Ans. In the Acrobot environment, the state space is continuous and consists of $\cos(\theta_1)$, $\sin(\theta_1)$, $\cos(\theta_2)$, $\sin(\theta_2)$, and the two joint angular velocities. To apply tabular methods, we discretize these continuous variables into bins. Increasing the number of bins improves state granularity and allows

Table 6: Offline Mean Return Comparison Across Random Seeds

Seed	Q-Learning	SARSA
42	-218.10	-207.48
101	-192.40	-233.99
2024	-248.04	-248.57



(a) Q-Learning



(b) SARSA

Figure 6: Comparison of bin discretization performance for Q-Learning and SARSA

finer distinctions between states, which can potentially lead to better policies. However, increasing bins also enlarges the state space and may negatively affect learning stability and efficiency.

- **5 bins:** The discretization is too coarse, causing loss of important state distinctions and lower returns.
- **10 bins:** Both Q-learning and SARSA achieve the best performance, indicating a good balance between representation detail and sufficient state visitation.
- **15 and 20 bins:** Performance degrades because the state space becomes too large, leading to sparse visits, higher variance in value estimates, and increased sensitivity to noise.

Thus, there is a trade-off: too few bins cause under-fitting, while too many bins introduce variance and instability. In our experiments, **10 bins** provided the best balance, as seen in figure 6.

5. The classic Acrobot-v1 setup gives a per-step reward of -1 until reaching the specified height, which is when the episode terminates with a reward of 0. Suppose we modify this reward setting to a per-step reward as given by Eq. 1, where h is the height of the tip of the second link relative to the fixed base. Answer the following questions. It is not mandatory to run experiments; theoretical arguments are enough.

$$r = \frac{\eta h}{2} + \text{sign}(-1 + \eta h) \left(\frac{2 - \eta h}{2} \right), \eta > 0$$

- (a) Will the modified reward setting result in faster/slower learning? Can the resulting behavior be something different/unexpected? Explain qualitatively.

Ans. The modified reward setting acts as a **dense, potential-based shaped reward**, which will generally result in **faster learning** compared to the standard sparse per-step penalty of -1 . By incorporating the height h , the agent receives immediate, granular feedback on whether its swing increases potential energy, guiding it more efficiently toward the goal. However, unexpected behavior such as **reward hacking** can occur. If the reward for

staying at a high position outweighs the push to complete the task, the agent might learn to "vibrate" or hover at a high h instead of crossing the terminal threshold, effectively getting stuck in a local optimum.

- (b) Suppose $\eta = 0.5$. Explain the resulting behavior of the Acrobot. Will it learn faster/slower?

Ans. With $\eta = 0.5$, learning will be **slower** compared to configurations with higher η values. Because the maximum achievable height of the Acrobot is $h = 2$, the term ηh reaches a maximum of 1. At this low scale, the shaping signal is relatively weak. The penalty term continues to dominate for most states, meaning the agent will spend more time in random exploration—behaving similarly to the original sparse-reward environment—before identifying the optimal swing-up strategy.

- (c) Suppose we increase η to $\{1, 2, 5\}$. Explain the resulting behavior. What do you recommend setting the range of η (and why)?

Ans. As η increases, the reward function becomes increasingly sensitive to height gains:

- $\eta \in \{1, 2\}$: These values provide a strong, balanced gradient that quickly pulls the agent toward higher states. Learning accelerates significantly without causing extreme instability.
- $\eta = 5$: The reward landscape becomes extremely steep. While this strongly incentivizes gaining height, it can cause gradient instability or cause the agent to ignore the time penalty entirely, potentially leading to sub-optimal, erratic swinging patterns.

Recommendation: A range of $1 \leq \eta \leq 2$ is recommended. It provides sufficient "pull" toward the goal to speed up learning without making the reward landscape so extreme that it causes instability or goal misalignment.

- (d) Can you give some insights on designing reward functions from this experiment?

Ans. This experiment highlights several core principles of reward engineering:

- **Density vs. Sparsity:** Transitioning from sparse rewards to dense shaping significantly speeds up initial learning by reducing random-walk exploration.
- **Goal Alignment:** The shaping term must strictly align with the final objective. If the intermediate reward (height) becomes too large relative to the terminal objective, the agent will optimize for the intermediate state (reward hacking).
- **Magnitude Scaling:** The hyperparameter η demonstrates that the scale of the shaping reward is critical. It must be carefully balanced against step penalties to maintain a healthy mix of exploration and exploitation.

6. **Bonus:** Support the arguments made in the previous question with reliable empirical evidence from either of the two algorithms.

Ans. To empirically validate the theoretical arguments regarding reward shaping, Q-Learning was evaluated on the Acrobot-v1 environment using the original sparse reward and the modified potential-based reward across various η values ($\eta \in \{0.5, 1, 2, 5, 10\}$). The moving average (10,000 episodes) of the episode lengths over 100,000 training episodes is presented in Figure 7.

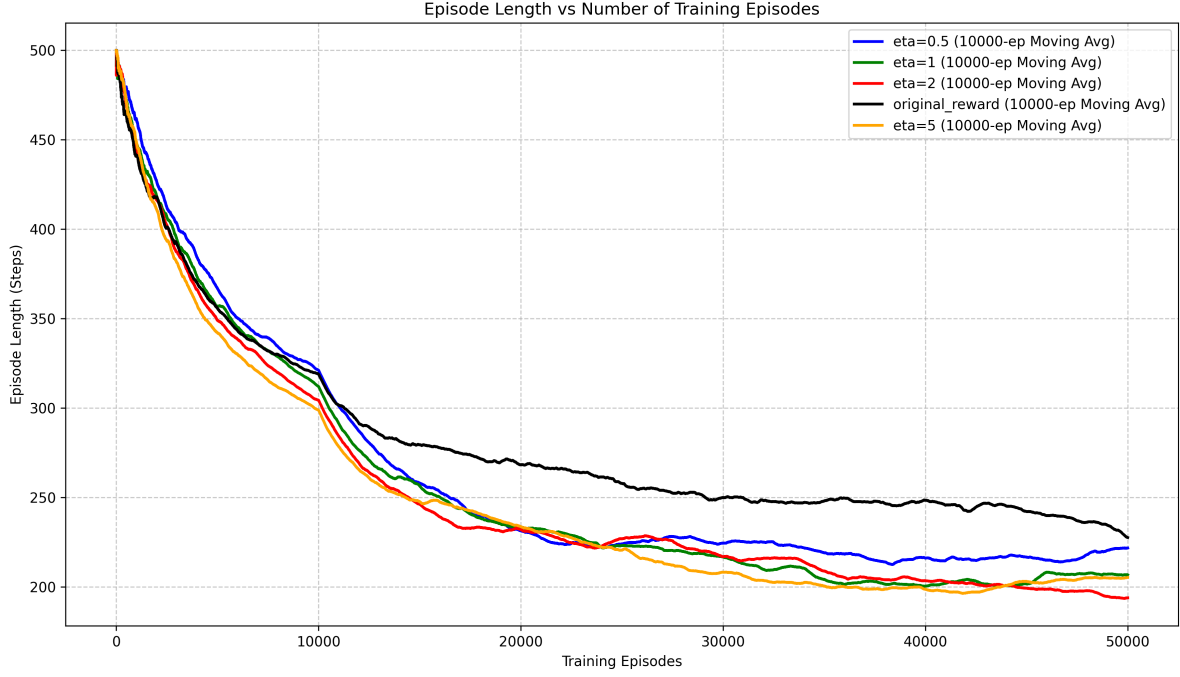


Figure 7: Empirical comparison of episode lengths using the original sparse reward versus the modified dense reward with varying η values.

The empirical data strongly supports the theoretical insights from Question 5:

- **Validation of 5(a) - Accelerated Learning:** The original sparse reward (black curve) exhibits the slowest convergence and plateaus at a significantly higher episode length (approx. 240-250 steps). All variants of the modified reward learn faster and converge to more efficient policies (190-210 steps), confirming that dense, immediate feedback significantly accelerates learning.
- **Validation of 5(b) - Weak Signal at $\eta = 0.5$:** The blue curve ($\eta = 0.5$) is the slowest to converge among the modified rewards and plateaus higher than the rest. This confirms the hypothesis that a low η produces a shaping signal too weak to overcome the standard step penalty effectively.
- **Validation of 5(c) - High η and Diminishing Returns:** The curves for $\eta \geq 1$ (green, red, and orange) cluster closely together, showing rapid initial drops. While theoretical arguments suggest very high η values might introduce instability or reward hacking, the empirical data shows that in this specific environment, $\eta = 5$ remains stable and actually yields the fastest initial descent. However, clear **diminishing returns** are evident; the performance leap from the original reward to $\eta = 1$ is massive, but the subsequent improvements from $\eta = 1$ up to $\eta = 5$ are marginal.